Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Отчёт

по научно-исследовательской работе студента

на тему Разработка методики и инструментов верификации соответствия TLA+ спецификации и исходного кода

ФИО студента: Железцов Никита Владимирович

Группа: ИУ8-114

Специальность: 10.05.01 «Компьютерная безопасность»

Специализация: 10.05.01_01 «Математические методы защиты информации»

Научный руководитель НИРС:

Работа выполнена:	_____	_____	Железцов Н.В.
	Дата	Подпись	(И. О. Фамилия)
Допуск к защите:	_____	_____	Медведев Н.В.
	Дата	Подпись	(И. О. Фамилия)

Дата защиты НИРС: _____

Результаты защиты: _____

Москва, 2026 г.

РЕФЕРАТ

Отчёт содержит 35 стр., 18 рис., 12 источн.

Ключевые слова: TLA+, трассировка, валидация трасс, модельная проверка, распределённые системы, Two-Phase Commit.

Основная цель работы — разработка и апробация подхода к валидации реализации распределённого алгоритма на основе формальной спецификации TLA+ с использованием анализа трасс исполнения.

В работе рассматривается задача сопоставления поведения реальной распределённой программы со спецификацией высокого уровня. В качестве объекта исследования выбран алгоритм двухфазной транзакции (Two-Phase Commit), широко применяемый для обеспечения согласованности в распределённых транзакционных системах. Реализован прототип алгоритма на языке C, включающий координатора (Transaction Manager), участников (Resource Managers) и сетевой сервер сообщений.

Для связывания кода со спецификацией разработан механизм инструментирования, позволяющий фиксировать изменения только выбранных переменных спецификации, а не полные состояния системы. Трассы исполнения формируются в формате NDJSON и затем автоматически объединяются и подаются на вход модели проверки TLC. Проверка корректности сводится к задаче ограниченной модельной проверки, где TLC анализирует соответствие полученной трассы спецификации TLA+.

В ходе экспериментов продемонстрировано обнаружение логической ошибки в реализации алгоритма, связанной с некорректным учётом сообщений от участников. Использование трассировки и формальной проверки позволило надёжно выявить расхождение между спецификацией и реализацией в условиях недетерминированной доставки сообщений.

СОДЕРЖАНИЕ

РЕФЕРАТ 4
СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ 6
ВВЕДЕНИЕ 7
ОСНОВНАЯ ЧАСТЬ 9
1 Теоретические основы 10
1.1 Формальная спецификация TLA+ 10
1.2 Модельная проверка трасс 11
2 Методика верификации 13
2.1 Задача валидации трасс 14
2.2 Реализация инструментирования спецификации 16
2.3 Библиотека инструментирования реализации 18
3 Демонстрационный пример: алгоритм двухфазной транзакции . 21
3.1 Реализация алгоритма на C 23
3.1.1 Менеджер ресурсов 23
3.1.2 Менеджер транзакций 24
3.2 Спецификация алгоритма на TLA+ 26
3.3 Инструментирование реализации 28
3.4 Проверка соответствия TLA+ спецификации и исходного кода 30
ЗАКЛЮЧЕНИЕ 33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 35

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения:

NDJSON — Newline Delimeted JSON.

RM — Resource Manager.

TLA — Temporal Logic of Actions.

TM — Transaction Manager.

WAL — Write-ahead log.

JSON — JavaScript Object Notation.

ВВЕДЕНИЕ

Распределённые системы лежат в основе современных вычислительных платформ, облачных сервисов и транзакционных приложений. Их корректность критически важна, поскольку ошибки в алгоритмах координации, согласования и обмена сообщениями могут приводить к потере данных, нарушению согласованности состояния и отказам сервисов. При этом поведение распределённых систем существенно усложняется из-за асинхронности, недетерминированности доставки сообщений и возможных сбоев отдельных узлов.

Формальные методы позволяют повысить уверенность в корректности распределённых алгоритмов за счёт построения строгих математических моделей. Одним из широко применяемых инструментов является язык спецификаций TLA+ [1], предназначенный для описания систем в терминах состояний и переходов между ними. С помощью средства проверки моделей TLC возможно исследование пространства состояний спецификации и доказательство инвариантов.

Однако формальная верификация спецификации не гарантирует корректность конкретной программной реализации алгоритма. На практике возникает разрыв между абстрактной моделью и исполняемым кодом. В данной работе рассматривается подход к сокращению этого разрыва путём сопоставления трасс исполнения реализации со спецификацией TLA+.

Актуальность темы исследования

Несмотря на широкое распространение формальных спецификаций в проектировании распределённых алгоритмов, проблема согласованности спецификации и реализации остаётся актуальной. Традиционные методы тестирования не обеспечивают полного покрытия возможных сценариев взаимодействия компонентов, особенно в условиях недетерминированного поведения сети.

Модельная проверка позволяет анализировать абстрактные модели, однако переход от модели к реальному коду часто сопровождается упрощениями, оптимизациями или изменением гранулярности атомарных действий. В результате возможны расхождения между поведением реализации и предполагаемой спецификацией.

Перспективным направлением является модельная валидация исполнения (Model-Based Trace Checking) [2], при которой реальные трассы работы распределённой программы проверяются на соответствие формальной спецификации с использованием средства модельной проверки. Такой подход позволяет обнаруживать ошибки реализации, избыточные ограничения спецификации и несоответствие уровня атомарности.

Разработка и практическая апробация метода валидации трасс исполнения по спецификации TLA+ представляется актуальной задачей, направленной на повышение надёжности распределённых систем. Этот метод активно применяется в индустрии [3].

Цель и задачи работы

Целью данной научно-исследовательской работы является разработка метода модельной валидации реализации распределённого алгоритма по формальной спецификации TLA+ на основе анализа трасс исполнения.

Для достижения поставленной цели были сформулированы следующие задачи:

а) Изучить теоретические основы языка спецификаций TLA+ и средства модельной проверки TLC.
б) Разработать механизм инструментирования реализации для регистрации изменений переменных спецификации.
в) Реализовать программный прототип распределённого алгоритма двухфазной транзакции.
г) Реализовать процедуру сопоставления трасс программы и спецификации с использованием TLC.
д) Провести экспериментальную валидацию и проанализировать полученные результаты.

Результатом работы является прототипный фреймворк, обеспечивающий сопоставление трасс распределённой реализации с формальной спецификацией TLA+, а также демонстрационная реализация алгоритма двухфазной транзакции, использованная для апробации предложенного метода.

ОСНОВНАЯ ЧАСТЬ

В основной части работы рассмотрены теоретические и практические аспекты верификации алгоритмов методом модельной проверки трасс с использованием TLA+ и TLC.

В разделе 1 изложены теоретические основы: описана модель вычислений в TLA+, каноническая форма спецификаций, а также принцип проверки трасс исполнения путём сопоставления их с поведениями формальной модели.

В разделе 2 представлена методика верификации. Формализована задача валидации трасс. Описана реализация ограниченной спецификации (`TraceSpec`), позволяющей свести задачу к ограниченной модельной проверке в TLC. Также разработана библиотека инструментирования на языке C для автоматической генерации трасс исполнения в формате NDJSON.

В разделе 3 методика продемонстрирована на примере алгоритма двухфазной транзакции. Представлены реализация менеджера ресурсов и менеджера транзакций на языке C, спецификация алгоритма на TLA+, а также процедура инструментирования кода. Проведённая валидация позволила обнаружить логическую ошибку в реализации, связанную с некорректной обработкой повторных сообщений `Prepared`, что подтверждает практическую применимость предложенного подхода.

1 Теоретические основы

В основе работы лежат методы формальной спецификации и модельной проверки распределённых систем. Формальные методы позволяют описывать систему в виде математической модели, задающей множество допустимых состояний и переходов между ними, а также формулировать инварианты и свойства корректности. Однако проверка спецификации не гарантирует корректность конкретной реализации. Для сопоставления поведения программы с её формальной моделью используется модельная валидация исполнения (Model-Based Trace Checking), позволяющая проверять реальные трассы исполнения относительно формальной спецификации

1.1 Формальная спецификация TLA+

TLA+ — язык спецификаций, основанный на теории множеств, логике первого порядка и темпоральной логике действий (англ. TLA, temporal logic of actions).

Темпоральная логика была введена Амиром Пнуэли в 1970-х годах [4]. Лесли Лэмпорт увидел недостаточность этой идеи для описания систем целиком и пришёл к мысли о необходимости использовать конечные автоматы, которым придавался смысл формул темпоральной логики, описывающих все возможные пути исполнения. Таким образом родилась идея темпоральной логики действий (TLA+), которая дополнила темпоральную логику следующим [5]:

- инвариантность при повторении состояния;
- темпоральное использование кванторов существования;
- принятие в качестве атомарных формул не только предикатов состояния, но и формул действий.

TLA-спецификация — темпоральная формула, часто называемая *Spec* и являющаяся предикатом (утверждением) о поведении. Поведение представляет собой возможный путь исполнения системы [6].

Состоянием называется присваивание значений переменных, шагом называется пара состояний. Теперь поведение можно представить как бесконечную последовательность состояний, а шагами поведения можно назвать пару последовательных состояний поведения. Предикатом состояния называется функция, результат которой — логическое значение истина или

ложь — соответствует утверждению о состоянии. Действием называется функция, имеющая смысл предиката над шагом. В этой функции участвуют как переменные первого шага, так и второго, которые обычно отмечаются штрихом [6].

$$Spec \triangleq Init \wedge \Box[Next]_{(v_1, v_2, \ldots, v_n)}$$

Здесь $Init$ — предикат состояния, $Next$ — действие, v_i — переменные, $\Box$ — единственный в данной спецификации темпоральный оператор (истинно во всех будущих состояниях).

TLC — это программа, которая по заданному TLA+ описанию системы и формулам свойств перебирает состояния системы и определяет, удовлетворяет ли система заданным свойствам.

Обычно работа с TLA+/TLC строится таким образом: описывается система в TLA+ (или уже реализованная, или та, которую собираются реализовать), формализуются интересные свойства, запускается TLC для проверки этих свойств. Однако данная работа ставит своей целью находить несоответствия работы программы и спецификации, для чего и будет использоваться TLC.

1.2 Модельная проверка трасс

Одним из часто возникающих вопросов при использовании TLA+ является проверка соответствия программной реализации формальной спецификации. В идеале хотелось бы доказать, что реализация является уточнением высокоуровневой спецификации, то есть что все возможные поведения программы допустимы спецификацией. Однако на практике такой подход сталкивается с серьёзными проблемами масштабируемости и применим лишь к очень небольшим программам.

Полная верификация (end-to-end verification) остаётся крайне дорогостоящей и технически сложной задачей вне зависимости от используемых инструментов. Даже в тех редких случаях, когда она осуществима, затраты времени и ресурсов делают её непрактичной для большинства прикладных систем [7].

Вместо этого в инженерной практике применяется более прагматичный подход — *model-based checking* или *model-based trace-checking* [2]. Его идея заключается не в доказательстве того, что *все* возможные поведения реализации соответствуют спецификации, а в проверке того, что *наблюдаемые* поведения ей соответствуют.

Подход состоит из следующих этапов:

а) Реализация инструментируется таким образом, чтобы фиксировать существенные события и изменения состояний.

б) Во время выполнения системы формируется трасса исполнения.

в) Полученная трасса интерпретируется как ограничение на поведение формальной спецификации.

г) Средство модельной проверки (в данном случае TLC) проверяет, существует ли поведение спецификации, согласованное с данной трассой.

Таким образом, задача сводится к проверке воспроизводимости наблюдаемой последовательности переходов в рамках формальной модели.

В отличие от классической модельной проверки, здесь не требуется перебирать всё пространство состояний системы. Проверяется лишь конкретная трасса, что существенно снижает вычислительную сложность. При этом метод применим к распределённым системам, где трассы могут собираться с нескольких узлов и агрегироваться для последующего анализа.

Данный подход имеет ряд преимуществ:

- низкая стоимость интеграции в существующий процесс разработки;
- возможность использования стандартных средств модельной проверки;
- обнаружение некорректных переходов даже при отсутствии явных сбоев выполнения;
- применимость к системам, написанным на различных языках программирования.

Следует отметить, что метод не является строгой формальной гарантией корректности, поскольку анализируется лишь конечное множество наблюдаемых трасс. Однако при достаточном объёме тестовых запусков вероятность обнаружения ошибок существенно возрастает. Как показывает практика применения формальных методов в индустрии [8], даже частичная формальная проверка позволяет обнаруживать серьёзные ошибки на ранних этапах разработки.

2 Методика верификации

Предлагаемая методика верификации основана на сопоставлении трасс исполнения распределённой реализации с формальной спецификацией, написанной на языке TLA+. Общая схема подхода представлена на рисунке 1.

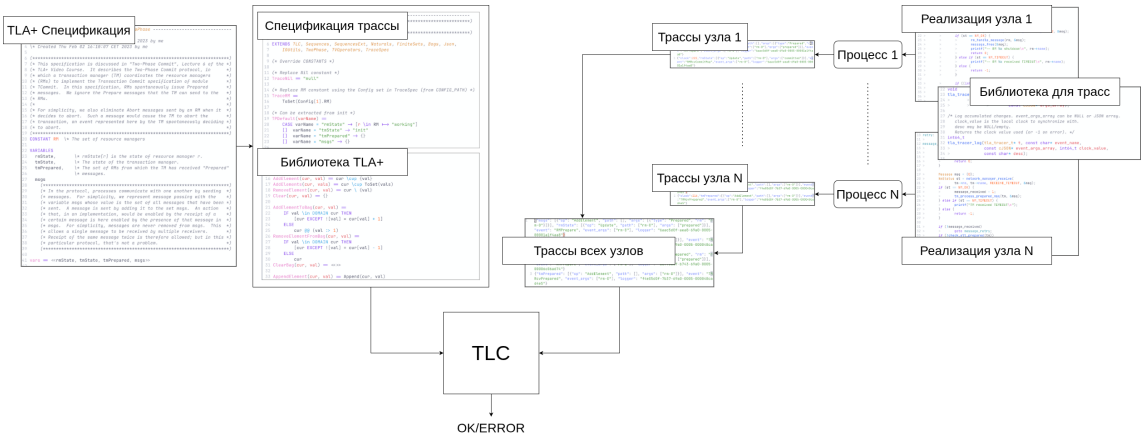

Рисунок 1 – Методика верификации

Исходными артефактами являются:

- программная реализация алгоритма.
- формальная спецификация этого алгоритма на языке TLA+.

Спецификация задаёт множество допустимых состояний системы и переходов между ними, однако не содержит сведений о конкретной реализации. Реализация, в свою очередь, выполняется в распределённой среде, где возможны различные порядки доставки сообщений и недетерминированные сценарии выполнения.

Для сопоставления поведения реализации со спецификацией используется инструментирование кода. В реализацию внедряется библиотека трассировки, которая фиксирует события, соответствующие переходам спецификации, а также изменения переменных спецификации. При выполнении системы каждый процесс (координатор и участники) формирует собственную трассу в формате JSON, содержащую последовательность операций обновления логических переменных модели.

Полученные локальные трассы агрегируются в единый глобальный файл, отражающий наблюдаемую последовательность изменений состояния всей распределённой системы. Таким образом, формируется абстрактное представление исполнения, выраженное в терминах переменных TLA+, а не в терминах низкоуровневых операций кода.

Для проверки корректности трассы используется дополнительная спецификация (Trace Specification), построенная на основе исходной TLA+ спецификации. Она расширяет её таким образом, чтобы поведение модели было ограничено наблюдаемой последовательностью изменений. Иными словами, трасса интерпретируется как частичное ограничение на допустимые переходы модели.

Далее средство модельной проверки TLC используется для проверки существования поведения спецификации, согласованного с данной трассой. Если TLC подтверждает корректность, это означает, что наблюдаемое исполнение допустимо спецификацией. В противном случае обнаруживается расхождение между реализацией и моделью, и TLC предоставляет контрпример, позволяющий локализовать проблему.

2.1 Задача валидации трасс

Задача валидации трасс формулируется следующим образом. Пусть S — множество конечных поведений, удовлетворяющих спецификации $Spec$, и пусть T — множество конечных поведений, представляемых трассой исполнения, где переменным, значения которых не были зафиксированы в трассе, разрешается принимать произвольные значения. Трасса считается совместимой со спецификацией тогда и только тогда, когда

$$S \cap T \neq \varnothing.$$

Иными словами, существует хотя бы одно поведение спецификации, согласующееся с наблюдаемой трассой. Заметим, что мы не проверяем условие уточнения (refinement) высокоуровневой спецификации трассой, выражаемое как $T \subseteq S$. Такое условие было бы слишком строгим в случае, когда значения некоторых переменных не фиксируются в трассе и могут выбираться недетерминированно.

Рис 2 иллюстрирует идею. Слева изображается линейная цепочка состояний, соответствующая трассе, полученной из инструментированной реализации. Справа — граф состояний, определяемый спецификацией TLA+. Требуется проверить, существует ли в графе состояний спецификации путь, который можно сопоставить данной трассе.

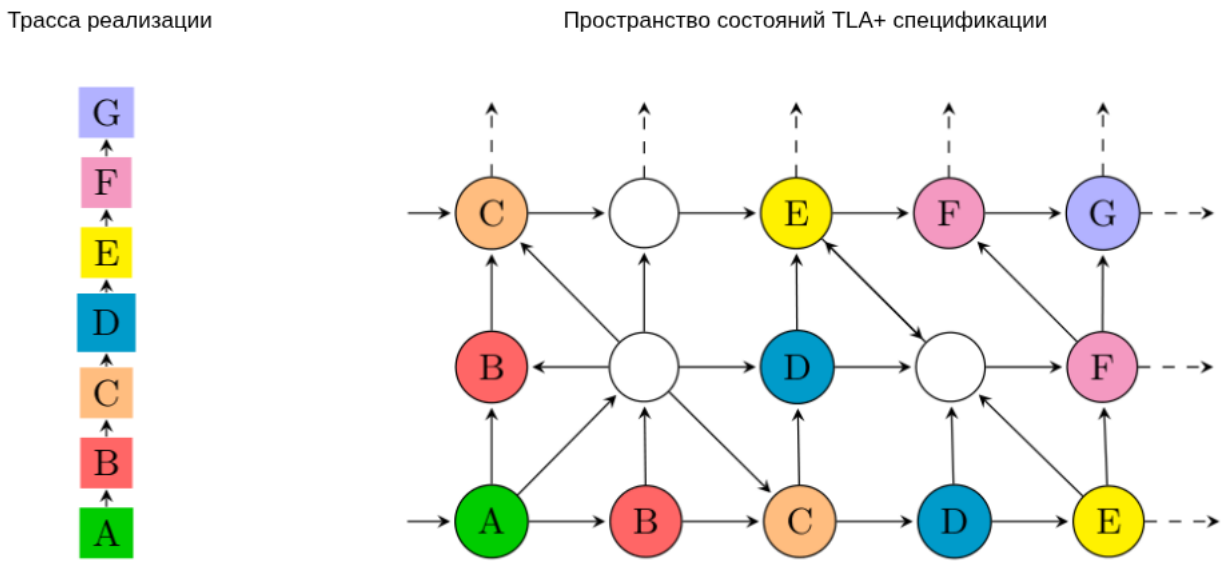

Рисунок 2 – Валидация трассы как поиск пути в пространстве состояний

Первое состояние трассы должно соответствовать некоторому начальному состоянию спецификации. Далее для каждого последующего шага трассы необходимо найти хотя бы одного преемника уже сопоставленного состояния, который соответствует следующему элементу трассы. Из-за неполной информации о значениях переменных одному шагу трассы может соответствовать несколько состояний спецификации. Если для некоторого состояния спецификации не существует допустимого продолжения, согласующегося со следующим элементом трассы, такая ветвь поиска отбрасывается. Трасса признаётся совместимой, если существует по крайней мере один путь в графе состояний спецификации, который полностью соответствует всей последовательности шагов трассы.

Следует учитывать дополнительную тонкость: атомарные шаги реализации не обязаны в точности совпадать с атомарными переходами спецификации. Один шаг программы может соответствовать нескольким шагам TLA+ или, наоборот, некоторым переходам спецификации может не соответствовать явного события в трассе. Поэтому задача валидации сводится к проверке существования пути в пространстве состояний спецификации при дополнительных ограничениях, задаваемых трассой. Практически эта редукция реализуется с помощью модельного проверяющего TLC.

В общем случае разработчик должен явно задать соответствие между именами событий в трассе и действиями спецификации, а также между именами логируемых переменных и переменными TLA+.

2.2 Реализация инструментирования спецификации

Множество S конечных поведений, удовлетворяющих исходной спецификации, задаётся формулой TLA+ $Spec$. Для того чтобы охарактеризовать пересечение $S \cap T$, то есть поведения спецификации, совместимые с трассой исполнения, к спецификации добавляются дополнительные ограничения. Таким образом, задача сводится к построению ограниченной спецификации, учитывающей информацию из трассы.

В разработанном фреймворке для этого используется отдельный модуль `TraceSpec`, предоставляющий операторы для построения ограниченной версии исходной спецификации. Его часть приведена на рис. 3.

```
1  ------------------- MODULE TraceSpec -------------------
2  EXTENDS Naturals, Sequences, TLC, Json, IOUtils
3
4  VARIABLE l
5  Trace == ndJsonDeserialize(IOEnv.TRACE_PATH)
6
7  IsEvent(e) ==
8      /\ l \in 1 .. Len(Trace)
9      /\ l' = l + 1
10     /\ "event" \in DOMAIN Trace[l]
11          => Trace[l].event = e
12     /\ UpdateVariables(Trace[l])
```

Рисунок 3 – Реализация $IsEvent()$ из модуля $TraceSpec$

Модуль объявляет переменную l, обозначающую номер текущей строки трассы. Оператор $Trace$ десериализует NDJSON-файл и представляет трассу программы в виде последовательности записей, каждая из которых соответствует одной записи журнала исполнения.

Оператор $IsEvent(e)$ инкапсулирует обработку текущей строки трассы. Он проверяет, что индекс l корректен, увеличивает его на единицу и, если в записи явно указано поле `"event"`, проверяет соответствие ожидаемому имени действия. Параметры события, если они заданы, учитываются далее. Кроме того, вызывается оператор $UpdateVariables$ (см. рис 4, который накладывает ограничения на значения переменных спецификации.

```
UpdateVariables(ll) ==
    /\ "rmState" \in DOMAIN ll
         => rmState' = UpdateVariable(rmState, "rmState", ll)
    /\ ...
    \* аналогично для tmState, tmPrepared, msgs
```

Рисунок 4 – Реализация $UpdateVariables()$

Данный оператор проверяет, присутствует ли в текущей строке трассы обновление соответствующей переменной. Если да, новое значение вычисляется с помощью вспомогательного оператора $UpdateVariable$ (также определенного в `TraceSpec`), который принимает старое значение переменной и описание операции из JSON-записи.

Для каждого действия исходной спецификации строится соответствующее действие ограниченной спецификации, получаемое конъюнкцией с предикатом $IsEvent$. Пример приведен на рис. 5

```
IsTMCommit ==
    IsEvent("TMCommit") /\ TMCommit
```

Рисунок 5 – Пример действия ограниченной спецификации

Поскольку TLC вычисляет формулы слева направо, сначала применяются ограничения из трассы (обновляются переменные), после чего проверяется, удовлетворяет ли полученное состояние предикату действия исходной спецификации. Для переменных, значения которых не зафиксированы в трассе, TLC может недетерминированно выбирать подходящие значения.

Для параметризованных действий учитываются аргументы, указанные в трассе. Например:

Таким образом, если в трассе указан конкретный параметр, рассматривается соответствующий экземпляр действия; иначе TLC разрешается выбрать любой допустимый параметр.

Общее отношение перехода ограниченной спецификации $TraceNext$ определяется как дизъюнкция всех таких ограниченных действий. Построение этой спецификации в большинстве случаев является систематическим и может быть автоматизировано.

При проверке TLC вычисляет все возможные преемники текущего состояния. Даже если из некоторого состояния нельзя продолжить сопоставление с трассой, в пространстве состояний могут существовать другие ветви, соответствующие трассе. Поэтому не следует включать проверку deadlock для ограниченной спецификации.

Для валидации трасс использовать постусловие, приведенное на рис. 6.

```
1 TraceAccepted ==
2     TLCGet("stats").diameter - 1 = Len(Trace)
```

Рисунок 6 – Условие валидации соответствия трасс

Если данное условие выполняется, существует поведение спецификации, полностью соответствующее трассе. Если нет, пересечение $S \cap T$ пусто, и трасса несовместима со спецификацией. В этом случае TLC не может предоставить единый контрпример (поскольку подходящего поведения не существует), однако может вывести максимальный префикс поведения спецификации, совпадающий с трассой.

Следует отметить, что при неполной трассировке возможны ложноположительные результаты: если трасса содержит слишком мало информации (например, фиксирует только длину исполнения без обновлений переменных и имён действий), модельный проверяющий может недетерминированно подобрать значения переменных так, что будет найдено совместимое поведение. Поэтому степень детализации трассировки напрямую влияет на точность валидации и размер пространства поиска.

2.3 Библиотека инструментирования реализации

Для сопоставления поведения реализации с формальной спецификацией TLA+ была разработана библиотека инструментирования на языке C [9]. Её задача — фиксировать изменения логических переменных модели и формировать трассу исполнения в формате NDJSON, пригодную для последующей валидации средствами TLC.

Библиотека реализует следующий принцип:

а) Во время выполнения программы фиксируются изменения логических переменных спецификации через `notify_change`.

б) В точке линейризации вызывается `log`, формируя атомарную запись трассы.

в) Полученный NDJSON-файл представляет последовательность логических шагов, которые могут быть сопоставлены с действиями TLA+ спецификации.

г) Полученные от разных процессов трассы можно объединить, используя Python скрипт `trace_merger.py`.

Библиотека предоставляет абстракцию `tla_tracer_t`, инкапсулирующую логику накопления изменений переменных и формирования записей трассы. Основной интерфейс представлен функциями, приведенными на рис 7.

```
typedef struct tla_tracer tla_tracer_t;

tla_tracer_t* tla_tracer_create(const char* trace_path, tlac_clock_t* clock);

void tla_tracer_destroy(tla_tracer_t* t);

void tla_tracer_notify_change(tla_tracer_t* t, const char* variable, const
>       >       >               cJSON* path_array, const char* op, const cJSON*
>       >       >               args_array);

int64_t tla_tracer_log(tla_tracer_t* t, const char* event_name, const cJSON*
>       >               event_args_array, int64_t clock_value, const char*
>       >               desc);
```

Рисунок 7 – Интерфейс библиотеки инструментирования реализации

Для согласования порядка событий используется абстракция часов (см. `tlac_clock_t` в функции `tla_tracer_create`). Часы могут быть:

- локальными (в памяти процесса),
- файловыми (для процессов на одном компьютере),

Все трассировщики синхронизируются через один и тот же механизм часов, что обеспечивает монотонность временных меток как внутри одного процесса, так и глобально между несколькими процессами. Возвращаемое функцией `tla_tracer_log` значение — это использованная временная метка.

Функция `tla_tracer_log` создаёт одну запись в файле трассы. В запись включаются (см. рис 8):

- временная метка (`"clock"`);
- все изменения переменных, накопленные после предыдущего вызова `log`;
- имя события (`"event"`);
- аргументы события (`"event_args"`, например, ID узла).

После записи буфер изменений очищается. Таким образом, один вызов `log` соответствует одному логическому шагу спецификации, объединяя несколько изменений переменных в атомарную запись трассы.

В реализации допускается логирование только подмножества переменных спецификации. В этом случае модельный проверяющий будет восстанавливать отсутствующую информацию недетерминированным образом. Тем не менее, более подробная трассировка (фиксация и переменных, и параметров действий) существенно сокращает пространство поиска TLC и повышает достоверность результатов валидации.

```
1  {
2    "title": "TraceEntry",
3    "description": "An entry in a trace",
4    "type": "object",
5    "properties": {
6      "clock": {
7        "description": "The timestamp of the event",
8        "type": "integer",
9        "minimum": 0
10     },
11     "additionalProperties": {
12       "type": "array",
13       "items": {
14         "type": "object",
15         "properties": {
16           "op": {
17             "type": "string"
18           },
19           "path": {
20             "type": "array"
21           },
22           "args": {
23             "type": "array"
24           }
25         },
26         "required": ["op", "path", "args"]
27       },
28       "minItems": 1
29     },
30     "event": {
31       "description": "Name of the event",
32       "type": "string"
33     },
34     "event_args": {
35       "type": "array",
36       "items": {
37         "type": "string"
38       }
39     }
40   },
41   "required": ["clock"]
42 }
```

Рисунок 8 – JSON структура записи трассы

3 Демонстрационный пример: алгоритм двухфазной транзакции

Протокол двухфазной транзакции (Two-Phase Commit, 2PC) представляет собой распределённый алгоритм атомарного подтверждения, предназначенный для координации нескольких процессов, участвующих в одной распределённой транзакции. Его задача заключается в том, чтобы все участники либо зафиксировали изменения (commit), либо откатили их (abort), обеспечивая тем самым атомарность выполнения.

Протокол использует выделенный процесс-координатор (Transaction Manager, TM) и множество участников (Resource Managers, RM). Координатор принимает окончательное решение о результате транзакции, а участники выполняют локальные действия и голосуют за подтверждение или отмену.

Работа протокола основана на следующих предположениях:

- у каждого узла имеется стабильное хранилище (WAL);
- узлы могут аварийно завершаться, но не выходят из строя навсегда;
- записи журнала не теряются и не повреждаются при сбое;
- любые два узла могут обмениваться сообщениями.

В отсутствие отказов выполнение транзакции состоит из двух фаз, оно представлено на рисунке 9.

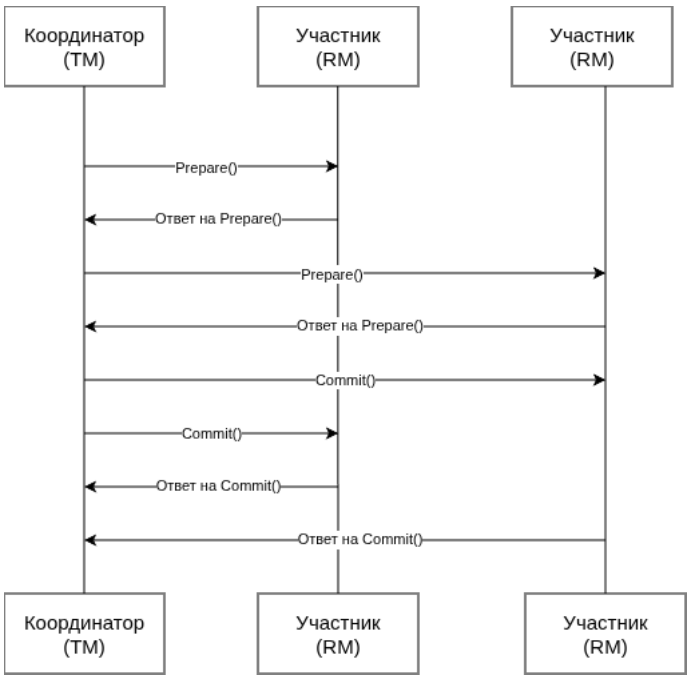

Рисунок 9 – Двухфазный комммит

1. Фаза запроса подтверждения (voting phase).

а) Координатор отправляет всем участникам сообщение `Prepare` (запрос на подтверждение).

б) Каждый участник выполняет локальную часть транзакции до точки фиксации.

в) Участник записывает соответствующие записи в журнал (undo/redo).

г) Участник отправляет координатору голос:

- `Yes` (готов зафиксировать), если локальная часть выполнена успешно;
- `No` (отказ), если обнаружена ошибка.

2. Фаза завершения (commit phase).

- Если координатор получил голос `Yes` от всех участников, он принимает решение `Commit`.
- В противном случае принимается решение `Abort`.

После принятия решения:

а) Координатор рассылает сообщение `Commit` или `Abort` всем участникам.

б) Каждый участник выполняет соответствующее действие: фиксирует изменения или откатывает их.

в) Участник освобождает ресурсы и отправляет подтверждение (ACK) координатору.

г) После получения всех подтверждений координатор завершает транзакцию.

Протокол обеспечивает атомарность: либо все участники фиксируют изменения, либо ни один из них не фиксирует. Однако 2PC является *блокирующим* протоколом. Если координатор выходит из строя после получения голосов `Yes`, участники, отправившие согласие, вынуждены ожидать решения, не имея возможности самостоятельно завершить транзакцию.

Кроме того, одновременный отказ координатора и одного из участников в фазе завершения может привести к неопределённости состояния до восстановления узлов. Несмотря на эти ограничения, протокол двухфазной транзакции широко используется благодаря своей простоте и строгой атомарной семантике [10].

3.1 Реализация алгоритма на C

В рамках работы был реализован прототип распределённого алгоритма двухфазной транзакции на языке C. Реализация состоит из двух типов процессов: менеджер транзакции (Transaction Manager, TM) и набор менеджеров ресурсов (Resource Manager, RM). Обмен сообщениями между процессами вынесен в отдельный компонент сетевого взаимодействия (`NetworkManager`) и происходит через центральный сервер сообщений (см. `server.c` в реализации [11]), который принимает команды отправки и выборки сообщений и хранит их в буфере.

Логика протокола соответствует высокоуровневой модели: каждый RM выполняет локальную работу, затем сообщает TM о готовности, а TM собирает готовности от всех RM и после этого принимает решение о завершении транзакции (в демонстрационном варианте — всегда `Commit`, а `Abort` возможен по таймауту).

3.1.1 Менеджер ресурсов

Основной цикл RM реализован в функции `rm_run` (фрагмент кода приведён на рис 10).

```
1  int rm_run(ResourceManager *rm) {
2  >       const int PROBABILITY_TO_ABORT = 100;
3  >       rm_working(rm);
4
5  >       while (1) {
6  >       >       if (rm_send_prepared(rm) != 0)
7  >       >       >       return -1;
8  >       >       Message msg = {0};
9  >       >       NmStatus st = network_manager_receive(
10 >       >       >       rm->nm, rm->name, rm->task_duration_ms * 2, &msg);
11 >       >       if (st == NM_OK) {
12 >       >       >       rm_handle_message(rm, &msg);
13 >       >       >       message_free(&msg);
14 >       >       >       return 0;
15 >       >       } else if (st == NM_TIMEOUT) {
16 >       >       } else {
17 >       >       >       return -1;
18 >       >       }
19 >       >       if ((int)(now_ms() - start) > ABORT_TIMEOUT) {
20 >       >       >       return 0;
21 >       >       }
22 >       }
23 }
```

Рисунок 10 – Реализация менеджера ресурсов

После старта процесс моделирует выполнение локальной операции (`rm_working`), затем переходит в фазу ожидания решения TM. Ключевой элемент протокола — повторная отправка сообщения о готовности при отсутствии ответа от TM: если по истечении таймаута ответа нет, RM повторяет отправку `Prepared`. Эта деталь важна, потому что в реальных распределённых системах сообщения могут теряться или задерживаться, и участникам необходимо обеспечивать прогресс протокола повторными попытками.

В реализации повторная отправка реализуется естественным образом за счёт цикла `while (1)`: на каждой итерации RM отправляет `Prepared`, затем вызывает `network_manager_receive` с конечным таймаутом. Если результат — `NM_OK`, значит TM прислал решение (`Commit` или `Abort`), и RM завершает работу после обработки сообщения. Если результат — `NM_TIMEOUT`, RM не меняет состояние и переходит к следующей итерации, тем самым повторяя отправку готовности.

Дополнительно предусмотрено аварийное завершение по общему дедлайну (`ABORT_TIMEOUT`): если RM слишком долго не получает решения, он прекращает работу (в прототипе это моделирует сбой или невозможность завершить транзакцию).

Важно отметить, что функция `rm_send_prepared` обновляет локальное состояние RM: первый раз при отправке `Prepared` RM переходит в состояние “prepared” и фиксирует соответствующее событие. Повторные отправки `Prepared` не меняют состояние RM, но создают ситуации, в которых TM может получить дубликаты сообщений.

3.1.2 Менеджер транзакций

Менеджер транзакций реализован в функции `tm_run` (приведена на рис 11). TM хранит список управляемых RM (`rm_names`) и множество RM, от которых уже получено `Prepared` (`prepared`). После запуска предусмотрена инициализационная задержка `init_duration_ms` — она позволяет дать RM время отправить первые сообщения `Prepared` прежде, чем TM начнёт интенсивно опрашивать сеть. Эта задержка удобна для экспериментов и повторяемости, хотя не является обязательной частью самого протокола.

```
int tm_run(TransactionManager *tm) {
>       const int RECEIVE_TIMEOUT = 100;
>       const int ABORT_TIMEOUT = 4000;

>       /* Initialising phase */
>       int64_t start = now_ms();
>       sleep_ms(tm->init_duration_ms);
>       int message_received;

retry:
>       message_received = 0;
message_retry:
>       if ((int)(now_ms() - start) > ABORT_TIMEOUT) {
>       >       (void)tm_abort(tm);
>       >       return 0;
>       }

>       Message msg = {0};
>       NmStatus st = network_manager_receive(
>       >       tm->nm, tm->name, RECEIVE_TIMEOUT, &msg);
>       if (st == NM_OK) {
>       >       message_received = 1;
>       >       tm_process_prepared_msg(tm, &msg);
>       } else if (st == NM_TIMEOUT) {
>       } else {
>       >       return -1;
>       }

>       if (!message_received)
>       >       goto message_retry;
>       if (!check_all_prepared(tm))
>       >       goto retry;

>       if (tm_commit(tm) != 0)
>       >       return -1;
>       return 0;
}
```

Рисунок 11 – Реализация менеджера транзакций

Далее TM входит в цикл ожидания сообщений. Получение сообщений осуществляется вызовом `network_manager_receive` с таймаутом `RECEIVE_TIMEOUT`. Если сообщение получено и оно содержит `Prepared`, TM добавляет отправителя в структуру `prepared`. После каждой успешно обработанной доставки проверяется условие завершения подготовительной фазы: `check_all_prepared`. Если условие истинно, TM рассылает всем RM сообщение `Commit` и завершает работу.

Если же сообщений долго нет и превышен общий дедлайн `ABORT_TIMEOUT`, TM выбирает условие `Abort` и уведомляет всех участников об отмене транзакции.

3.2 Спецификация алгоритма на TLA+

В качестве формальной модели используется спецификация алгоритма двухфазной транзакции (Two-Phase Commit) на языке TLA+. Данная спецификация основана на публичном примере из репозитория TLA+ Examples [12] и приведена в сокращенном виде на рис. 12.

```
1  CONSTANT RM
2  VARIABLES rmState, tmState, tmPrepared, msgs
3  vars == << rmState, tmState, tmPrepared, msgs >>
4  Messages == [type : {"Prepared"}, rm : RM] \cup [type : {"Commit", "Abort"}]
5
6  TPInit ==
7      /\ rmState = [r \in RM ⟼ "working"] /\ tmState = "init"
8      /\ tmPrepared = {} /\ msgs = {}
9
10 RMPrepare(r) ==
11     /\ rmState[r] = "working"
12     /\ rmState' = [rmState EXCEPT ![r] = "prepared"]
13     /\ msgs' = msgs \cup { [type ⟼ "Prepared", rm ⟼ r] }
14
15 RMRcvCommitMsg(r) ==
16   /\ [type ⟼ "Commit"] \in msgs
17   /\ rmState' = [rmState EXCEPT ![r] = "committed"]
18
19 TMRcvPrepared(r) ==
20     /\ tmState = "init"
21     /\ [type ⟼ "Prepared", rm ⟼ r] \in msgs
22     /\ tmPrepared' = tmPrepared \cup {r}
23
24 TMCommit ==
25     /\ tmState = "init" /\ tmPrepared = RM
26     /\ tmState' = "done" /\ msgs' = msgs \cup { [type ⟼ "Commit"] }
27
28 (* RMRcvAbortMsg == ... *)
29 (* TMAbort == ... *)
30
31 TPNext ==
32     \/ TMCommit \/ TMAbort
33     \/ \E r \in RM :
34          \/ RMPrepare(r)
35          \/ TMRcvPrepared(r)
36
37 Spec == TPInit /\ [][TPNext]_vars
38
39 Consistent ==
40     \A r1, r2 \in RM : ~(rmState[r1] = "aborted" /\ rmState[r2] = "committed")
```

Рисунок 12 – Спецификация алгоритма двухфазной транзакции

Спецификация описывает взаимодействие менеджера транзакции (TM) и множества менеджеров ресурсов (RM), которые должны согласованно принять решение о фиксации (commit) или откате (abort) транзакции. Менеджеры ресурсов отправляют сообщения о готовности выполнить транзакцию, а менеджер транзакции, получив голоса всех участников, принимает итоговое решение и рассылает соответствующее сообщение.

Модуль сначала объявляет константу `RM`, представляющую множество идентификаторов менеджеров ресурсов, и четыре переменные состояния:

- `rmState` — функция, отображающая каждый RM в его локальное управляющее состояние;
- `tmState` — состояние менеджера транзакции;
- `tmPrepared` — множество RM, объявивших готовность;
- `msgs` — множество сообщений, отправленных в системе.

Начальное состояние задаётся предикатом `TPInit`: все RM находятся в состоянии `"working"`, TM — в состоянии `"init"`, а множества `tmPrepared` и `msgs` пусты.

Далее определяются действия, описывающие переходы системы. Например, действие `RMPrepare(r)` моделирует объявление RM с идентификатором `r` своей готовности: его состояние изменяется на `"prepared"`, а в множество сообщений добавляется соответствующее сообщение.

Действие `TPNext` задаёт отношение перехода системы как дизъюнкцию всех возможных действий. Формула `Spec` объединяет начальное условие и отношение перехода, задавая полную спецификацию поведения.

Свойства спецификации могут проверяться средствами TLC и TLAPS. В частности, инвариант `Consistent` утверждает, что не существует двух RM, один из которых завершил транзакцию успешно, а другой — отменил.

Затем с помощью модуля `TraceSpec` была разработана специализированная обёртка над спецификацией алгоритма двухфазной транзакции — модуль `TwoPhaseTrace`. Его задача заключается в связывании конкретной спецификации `TwoPhase` с механизмом обработки трасс: поведение спецификации ограничивается конкретной последовательностью событий, полученной из исполнения программы. Ее сокращенная версия представлена на рис. 13.

```
1  ---------------------------- MODULE TwoPhaseTrace ----------------------------
2  (***************************************************************************)
3  (* Trace spec that refines 2PC *)
4  (***************************************************************************)
5
6  EXTENDS TLC, Sequences, SequencesExt, Naturals, FiniteSets, Bags, Json,
7          IOUtils, TwoPhase, TVOperators, TraceSpec
8
9  (* Can be extracted from init *)
10 TPDefault(varName) ==
11     CASE varName = "rmState" -> [r \in RM |-> "working"]
12     []  varName = "tmState" -> "init"
13     []  varName = "tmPrepared" -> {}
14     []  varName = "msgs" -> {}
15
16 TPUpdateVariables(t) ==
17     /\
18         IF "rmState" \in DOMAIN t
19         THEN rmState' = UpdateVariable(rmState, "rmState", t)
20         ELSE TRUE
21     (*...*)
22
23 (* Predicate actions *)
24 IsTMCommit ==
25     /\ IsEvent("TMCommit")
26     /\ TMCommit
27
28 (* ... *)
29
30 TPTraceNext ==
31         \/ IsTMCommit
32         (* ... *)
33 ===============================================================================
```

Рисунок 13 – Обертка над спецификацией алгоритма для верификации трасс

3.3 Инструментирование реализации

После разработки реализации и спецификации алгоритма двухфазной транзакции следующим шагом стало инструментирование этой реализации. Цель инструментирования — установить явное соответствие между действиями кода на языке C и переходами TLA+ спецификации (`RMPrepare`, `TMRcvPrepared`, `TMCommit`, `RMRcvCommitMsg` и др.).

Инструментирование выполнялось в *точках линеаризации* — местах, где реализация завершает логически атомарный шаг протокола и обновляет разделяемое состояние (например, изменение локального состояния RM или добавление сообщения в множество `msgs`). В этих точках:

а) регистрируются изменения переменных спецификации через `notify_change`;

б) выполняется вызов `tracer_log`, связывающий накопленные изменения с конкретным действием TLA+.

```c
static void notify_change(tla_tracer_t* t,
                          const char* var,
                          const char* field,
                          const char* op,
                          const char* kind,
                          const char* s1,
                          const char* s2)
{
    cJSON* path = cJSON_CreateArray();
    cJSON* args = cJSON_CreateArray();

    if (field)
        cJSON_AddItemToArray(path, cJSON_CreateString(field));

    if (kind && strcmp(kind, "string") == 0)
        cJSON_AddItemToArray(args, cJSON_CreateString(s1));
    else if (kind && strcmp(kind, "object_type") == 0) {
        cJSON* obj = cJSON_CreateObject();
        cJSON_AddStringToObject(obj, "type", s1);
        cJSON_AddItemToArray(args, obj);
    }
    else if (kind && strcmp(kind, "object_type_rm") == 0) {
        cJSON* obj = cJSON_CreateObject();
        cJSON_AddStringToObject(obj, "type", s1);
        cJSON_AddStringToObject(obj, "rm", s2);
        cJSON_AddItemToArray(args, obj);
    }

    tla_tracer_notify_change(t, var, path, op, args);
    cJSON_Delete(path);
    cJSON_Delete(args);
}
```

Рисунок 14 – Вспомогательная функция для инструментирования алгоритма

Для удобства использования библиотеки была реализована вспомогательная функция `notify_change`, упрощающая формирование JSON-структур `path` и `args`. На рис 14 приведена сокращённая версия функции, отражающая её логическую структуру:

Рассмотрим инструментирование отправки сообщения `Prepared` менеджером ресурсов. В реализации эта логика сосредоточена в функции `rm_send_prepared`, которая приведена на рис 15

```c
static int rm_send_prepared(ResourceManager *rm) {
>       if (rm->state != RM_PREPARED) {
>       >       rm->state = RM_PREPARED;
>       >       notify_change(rm->tracer, "rmState", rm->name, "Update",
>       >       >             "string", "prepared", NULL);
>       >       notify_change(rm->tracer, "msgs", NULL, "AddElement",
>       >       >             "object_type_rm", MSG_PREPARED, rm->name);
>       >       tracer_log(rm->tracer, "RMPrepare", rm->name);
>       }

>       Message m = message_make(rm->name, rm->tm_name, MSG_PREPARED, 0);
>       network_manager_send(rm->nm, &m);
>       message_free(&m);
>       return 0;
}
```

Рисунок 15 – Реализация отправки сообщения prepared

Для RM с именем `"rm-0"` соответствующая запись трассы имеет вид, приведенный на рис. 16

```
1  {
2    "clock": 4,
3    "rmState": [
4      { "op": "Update",
5        "path": ["rm-0"],
6        "args": ["prepared"] }
7    ],
8    "msgs": [
9      { "op": "AddElement",
10       "path": [],
11       "args": [
12         { "type":"Prepared", "rm":"rm-0" }
13       ]
14     }
15   ],
16   "event": "RMPrepare",
17   "event_args": ["rm-0"]
18 }
```

Рисунок 16 – Полученное сообщение трассы

3.4 Проверка соответствия TLA+ спецификации и исходного кода

В процессе сопоставления реализации алгоритма двухфазной транзакции со спецификацией на TLA+ был выявлен дефект в реализации, связанный с обработкой сообщений `Prepared` менеджером транзакции (TM).

В корректной реализации TM должен фиксировать множество менеджеров ресурсов, от которых получены сообщения `Prepared`, и принимать решение о фиксации транзакции только тогда, когда

$$tmPrepared = RM,$$

то есть когда каждый участник проголосовал за выполнение транзакции.

Однако возможна упрощённая реализация, в которой вместо множества хранится лишь счётчик полученных сообщений `Prepared`. В этом случае условие готовности проверяется сравнением счётчика с числом менеджеров ресурсов. Такая реализация работает корректно, если сообщения не теряются и не дублируются.

Тем не менее, в нашей системе RM при истечении таймаута повторно отправляет сообщение `Prepared`. Если TM не проверяет уникальность отправителя, повторное сообщение от одного и того же RM может быть учтено несколько раз. Это приводит к преждевременной фиксации, когда ещё не все участники объявили готовность.

Дефект был выявлен при первом запуске `run_pipeline_until_fail.sh`. В журнале исполнения видно, что RM `rm-0` несколько раз отправляет сообщение `Prepared` из-за тайм-аута (см. рис. 17).

```
1 TM received Prepared from rm-0 ⇒ {rm-0}
2 ...
3 TM received Prepared from rm-0 ⇒ {rm-0, rm-0}
4 ...
5 TM received Prepared from rm-0 ⇒ {rm-0, rm-0, rm-1, rm-0}
```

Рисунок 17 – Повторное получение сообщения от rm-0

В результате TM принимает решение `Commit`, несмотря на то что RM `rm-3` ещё не отправил сообщение о готовности. Это видно из трассы: событие `TMCommit` происходит до того, как TM получит `Prepared` от `rm-3`. Фрагмент NDJSON-трассы приведен на рис. 18.

```
1 {"tmState": [{"op": "Update", "path": [], "args": ["done"]}],
2  "msgs": [{"op": "AddElement", "path": [],
3            "args": [{"type": "Commit"}]}],
4  "event": "TMCommit"}
5 ...
6 {"msgs": [{"op": "AddElement", "path": [],
7            "args": [{"type": "Prepared", "rm": "rm-3"}]}],
8  "rmState": [{"op": "Update", "path": ["rm-3"],
9               "args": ["prepared"]}],
10 "event": "RMPrepare"}
```

Рисунок 18 – Часть трассы, иллюстрирующая некорректное поведение TM

То есть подтверждение транзакции произошло до получения готовности от всех RM, что противоречит предикату `TMCommit` в спецификации.

При проверке трассы TLC ограниченная спецификация `TwoPhaseTrace` не смогла сопоставить событие `TMCommit` с допустимым переходом исходной модели. Это означает, что в пространстве состояний спецификации не существует поведения, в котором переход `TMCommit` возможен при текущем значении переменной `tmPrepared`.

Таким образом, модельная валидация трасс позволила надёжно обнаружить логическую ошибку реализации, которая могла бы проявляться лишь при определённых сетевых сценариях (повторная отправка сообщений). После исправления реализации (замена счётчика на множество уникальных RM) трассы стали успешно проходить проверку TLC.

ЗАКЛЮЧЕНИЕ

В работе был разработан и реализован подход к верификации соответствия TLA+ спецификации и исходного кода. В основе предложенного метода лежит сопоставление трасс реального исполнения программы с допустимыми поведениями формальной модели при помощи TLC.

В теоретической части были рассмотрены основы формальной спецификации в TLA+, каноническая форма записи алгоритмов в виде машин состояний, а также формальная постановка задачи валидации трасс как проверки непустоты пересечения множества поведений спецификации и множества поведений, совместимых с наблюдаемой трассой исполнения. Показано, что задача может быть сведена к ограниченной модельной проверке посредством построения обёртки над исходной спецификацией.

В практической части была реализована библиотека инструментирования на языке C, позволяющая логировать изменения абстрактных переменных спецификации и связывать их с действиями TLA+. Разработана инфраструктура для автоматической генерации трасс в формате NDJSON и их последующей проверки с использованием TLC. Для спецификации алгоритма была создана ограниченная версия модели (`TwoPhaseTrace`), обеспечивающая учёт ограничений, задаваемых трассой исполнения.

Предложенная методика была апробирована на демонстрационном примере — алгоритме двухфазной транзакции. Реализация менеджера транзакции и менеджеров ресурсов была инструментирована и проверена на соответствие формальной спецификации. В ходе валидации был обнаружен дефект реализации, связанный с некорректной обработкой повторных сообщений `Prepared`, приводящий к преждевременной фиксации транзакции. Обнаружение ошибки средствами трассовой верификации подтверждает практическую ценность разработанного подхода.

Основным преимуществом предложенного метода является его применимость к существующему коду без необходимости полной формальной верификации реализации на уровне исходного или машинного кода. Подход позволяет выявлять логические ошибки проектирования и расхождения между моделью и реализацией при относительно невысокой стоимости внедрения. При этом степень детализации трассировки может варьироваться, что обеспечивает баланс между точностью проверки и вычислительными затратами.

К ограничениям метода относится его неполнота: анализируется лишь подмножество поведений, соответствующее наблюдаемым трассам. Тем не менее, при достаточном покрытии сценариев исполнения данный подход существенно повышает уверенность в корректности реализации.

Таким образом, в работе показано, что интеграция формальной спецификации TLA+ с трассовой валидацией исполнения представляет собой эффективный и практически применимый способ повышения надёжности распределённых программных систем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lamport L. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. — Boston, MA : Addison-Wesley, 2002.

2. Model-Based Trace-Checking / Y. Howard [и др.] // arXiv preprint arXiv:1111.2825. — 2011. — Нояб. — arXiv: 1111.2825 [cs.SE].

3. How Amazon Web Services Uses Formal Methods / C. Newcombe [и др.] // Communications of the ACM. — 2015. — Март. — Т. 58, № 4. — С. 66—73. — DOI: 10.1145/2699417.

4. Amir Pnueli. The temporal logic of programs // In Proceedings of the 18th Annual Symposium on the Foundations of Computer Science. — 1977. — С. 46—57.

5. Lamport L. The Specification Language TLA+. — 2008. — С. 616—620.

6. Henri Habrias, Marc Frappier. Software Specification Methods. — John Wiley and Sons, 2006. — С. 418.

7. Validating Traces of Distributed Programs Against TLA+ Specifications / H. Cirstea [и др.] // Software Engineering and Formal Methods: 22nd International Conference, SEFM 2024. — Aveiro, Portugal : Springer, 11.2024. — С. 126—143. — DOI: 10.1007/978-3-031-77382-2_8.

8. Smart Casual Verification of CCF's Distributed Consensus and Consistency Protocols / H. Howard [и др.] // arXiv preprint arXiv:2406.17455. — 2024. — arXiv: 2406.17455 [cs.DC].

9. Железцов Н. tlac: C библиотека для трассировки кода. — 2025. — URL: https://github.com/Serpentian/tlac (дата обращения 15.09.2025).

10. Bernstein P. A., Newcomer E. Principles of Transaction Processing. — 2-е изд. — Morgan Kaufmann, 2009. — ISBN 978-1-55860-623-4. — Ч. 8.

11. Железцов Н. two-phase-tlac: реализации алгоритма двухфазного коммита для валидации соответствия TLA+ спецификации. — 2025. — URL: https://github.com/Serpentian/two-phase-tlac (дата обращения 15.11.2025).

12. TLA+ Examples Repository. Two-Phase Commit Specification. — 2024. — Accessed: 2025-11-8. https://github.com/tlaplus/Examples/tree/master/specifications/transaction_commit.